

Классы в C#

C#

C# является полноценным объектно-ориентированным языком. Это значит, что программу на C# можно представить в виде взаимосвязанных взаимодействующих между собой объектов.

Пример

Описанием объекта является **класс**, а объект представляет **экземпляр этого класса**. Можно еще провести следующую аналогию. У нас у всех есть некоторое представление о человеке, у которого есть имя, возраст, какие-то другие характеристики. То есть **некоторый шаблон** - этот шаблон можно назвать **классом**. **Конкретное воплощение** этого шаблона может отличаться, например, одни люди имеют одно имя, другие - другое имя. И реально существующий человек (фактически экземпляр данного класса) будет представлять **объект** этого класса.

Пустой класс

```
//пустой класс животное
```

```
Ссылка: 0
```

```
class Animal
```

```
{
```

```
...
```

```
}
```

Поля и методы класса

Класс может хранить некоторые данные. Для хранения данных в классе применяются **поля (свойства)**. По сути **поля класса** - это переменные, определенные на уровне класса.

```
class Animal
{
    public string name; //имя
    public int age; //возраст
    public int legsCount; //кол-во ног
}
```

Поля и методы класса

Кроме того, класс может определять некоторое поведение или выполняемые действия. Для определения поведения в классе применяются **методы**.

```
class Animal
{
    public string name; //имя
    public int age; //возраст
    public int legsCount; //кол-во ног
    Ссылки: 0
    public void move() //метод для передвижения
    {
        Console.WriteLine($"{name} передвигается на своих {legsCount}-х ногах");
    }
}
```


Создание объекта класса

После определения класса мы можем создавать его объекты. Для создания объекта применяются **конструкторы**. По сути конструкторы представляют специальные методы, которые называются так же как и класс, и которые вызываются при создании нового объекта класса и выполняют инициализацию объекта.

Конструктор по умолчанию

Если в классе не определено ни одного конструктора (как в случае с нашим классом `Animal`), то для этого класса автоматически создается пустой конструктор по умолчанию, который не принимает никаких параметров.

Создание объекта класса Animal

```
Animal animal = new Animal();
```

Ссылка: 2

```
class Animal
```

```
{
```

```
    public string name; //имя
```

```
    public int age; //возраст
```

```
    public int legsCount; //кол-во ног
```

Ссылка: 0

```
    public void move() //метод для передвижения
```

```
    {
```

```
        Console.WriteLine($"{name} передвигается на своих {legsCount}-х ногах");
```

```
    }
```

```
}
```

```
Animal animal = new Animal();  
Console.WriteLine(animal.name);  
Console.WriteLine(animal.age);  
Console.WriteLine(animal.legsCount);
```

Ссылка: 2

```
class Animal  
{  
    public string name; //имя  
    public int age; //возраст  
    public int legsCount; //кол-во ног  
    Ссылка: 0  
    public void move() //метод для передвижения  
    {  
        Console.WriteLine($"{name} передвигается на своих {legsCount} ногах");  
    }  
}
```

Консоль отладки Microsoft Visual Studio

0

0

C:\Users\NE01\Documents\classes.exe (пр
Нажмите любую к

Добавление конструктора с параметрами

Для того чтобы при создании объекта мы могли указать у него значения полей (свойств) нужно добавить в класс какой-нибудь другой конструктор, кроме конструктора по умолчанию.

```
class Animal
{
    public string name;
    public int age;//возраст
    public int legsCount;//кол-во ног
    //конструктор с параметрами (при создании надо указать имя, возраст, кол-во ног)
    Ссылка: 1
    public Animal(string name, int age, int legsCount)
    {
        this.name = name;
        this.age = age;
        this.legsCount = legsCount;
    }
    Ссылка: 0
    public void move()//метод для передвижения
    {
        Console.WriteLine($"{name} передвигается на своих {legsCount}-х ногах");
    }
}
```

Обращение к функциональности класса

Для обращения к функциональности класса - полям, методам (а также другим элементам класса) применяется точечная нотация точки - после объекта класса ставится точка, а затем элемент класса:

объект.поле_класса

объект.метод_класса(параметры_метода)

```
//Создание объекта класса Animal - cat  
Animal cat = new Animal("Кот", 12, 4);  
Console.WriteLine($"Имя: {cat.name}");  
Console.WriteLine($"Возраст: {cat.age}");  
Console.WriteLine($"Количество ног: {cat.legsCount}");
```

Ссылка: 3

Консоль отладки Microsoft Visual Studio

Имя: Кот
Возраст: 12
Количество ног: 4

```
cat.move();
```

Кот передвигается на своих 4-х ногах

Наследование в C#

Наследование

Наследование является одним из ключевых моментов ООП. Благодаря наследованию один класс может унаследовать функциональность другого класса.

Наследование

У нас есть класс `Animal` (Животное). Но такого животного, как животное не существует, зато есть конкретные виды — коты, собаки, птицы и т. д. И все животные имеют имя, возраст, способность передвигаться и прочие общие моменты. Поэтому вместо того чтобы создавать под каждый тип животного отдельный класс, мы будем использовать наследование.

Наследование

Родительский класс — это класс, от которого другой класс наследует свойства и методы.

Дочерний класс — это класс, который наследует функциональность от родительского класса и может добавлять свои собственные или изменять существующие методы и атрибуты.

В таком случае `Animal` — это родительский, а, например, `Cat` и `Dog` — дочерние.

Наследование

После двоеточия мы указываем **родительский класс** для данного класса. Для класса Cat родительским является Animal, и поэтому класс Cat наследует все те же свойства, методы, поля, которые есть в классе Animal. Единственное, что не передается при наследовании, это конструкторы базового класса с параметрами.

```
//класс Cat - наследник класса Animal
```

```
//Наследует все, кроме конструктора
```

```
Ссылка: 0
```

```
class Cat : Animal
```

```
{
```

```
  |  
  |  
  |
```

```
}
```

//класс Cat - наследник класса Animal

//Наследует все, кроме конструктора

Ссылка: 1

```
class Cat : Animal
```

```
{
```

```
    public double whiskerLength; //длина усов
```

```
    //конструктор для создания кота
```

Ссылка: 0

```
    public Cat(string name, int age, int legsCount, double whiskerLength) : base(name, age, legsCount,
```

```
    {
```

```
        Console.WriteLine("Кот создан");
```

```
        this.whiskerLength = whiskerLength;
```

```
    }
```

```
}
```

Наследование

Класс `Animal` имеет конструктор, который устанавливает свойства `name`, `age`, `legsCount`. Поскольку класс `Cat` наследует и устанавливает то же свойства, то логично было бы не писать по сто раз код установки, а как-то вызвать соответствующий код класса `Animal`.

С помощью ключевого слова **base** мы можем обратиться к базовому классу. В нашем случае в конструкторе класса `Cat` нам надо установить длину усов. Но имя, возраст, кол-во ног мы передаем на установку в конструктор базового класса, то есть в конструктор класса `Animal`, с помощью выражения `base`.


```
Cat cat = new Cat ()
```



```
Cat.Cat(string name, int age, int legsCount, double whiskerLength)
```

Ссылка: 3

class Animal

```
//создание объектов класса Cat
```

```
Cat munchkin = new Cat("Манчкин", 2, 4, 6.5);
```

```
Cat lion = new Cat("Лев", 8, 4, 15);
```

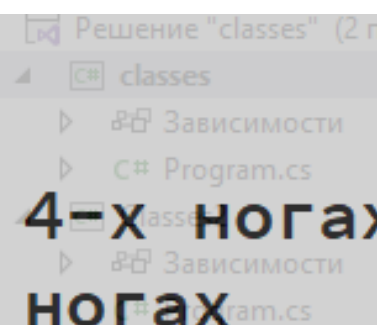
```
//создание объектов класса Cat
Cat munchkin = new Cat("Манчкин", 2, 4, 6.5);
Cat lion = new Cat("Лев", 8, 4, 15);
munchkin.move();
lion.move();
```

Кот создан

Кот создан

Манчкин передвигается на своих 4-х ногах

Лев передвигается на своих 4-х ногах



Задание

Создать класс `Person`(человек) с полями: имя, возраст. Добавить в класс метод, выводящий информацию о человеке (имя, возраст) и конструктор с параметрами. Создать экземпляр класса `Person` и вывести информацию о нем.

Добавить в программу дочерний класс `Worker` (работник), наследник `Person`. Добавить поле компания. Добавить конструктор с параметрами. Создать экземпляр класса и вывести информацию о работнике.

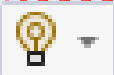
Позвать преподавателя для проверки.

Абстрактные классы в C#

Абстрактные классы

Абстрактный класс похож на обычный класс. Он также может иметь переменные, методы, конструкторы, свойства. Единственное, что при определении абстрактных классов используется ключевое слово `abstract`.

```
Animal animal = new Animal();
```



`Animal.Animal(string name, int age, int legsCount)`

CS0144: Не удается создать экземпляр абстрактного типа или интерфейса "Animal"

IDE0090: выражение new можно упростить

Показать возможные решения (Alt+BВВВилиCtrl+.)

Ссылка: 5

```
abstract class Animal
```

```
{
```

```
    public string name;
```

```
    public int age; //1
```

```
    public int legsCount;
```

```
    //конструктор с параметрами (при создании надо указать имя, возраст, кол-во ног)
```

Ссылка: 2

```
    public Animal(string name, int age, int legsCount)
```

```
{
```

```
        this.name = name;
```

```
        this.age = age;
```

```
        this.legsCount = legsCount;
```

```
}
```

//класс Собака, наследник класса Животное

Ссылка: 1

```
class Dog : Animal
```

```
{
```

```
    public string breed;//порода
```

```
    //конструктор для создания собаки
```

Ссылка: 0

```
    public Dog(string name, int age, int legsCount, string breed) : base(name, age, legsCount)
```

```
    {
```

```
        Console.WriteLine("Собака создана");
```

```
        this.breed = breed;
```

```
    }
```

Ссылка: 0

```
    public void barking()
```

```
    {
```

```
        Console.WriteLine("гав гав..");
```

```
    }
```

```
}
```

```
Dog husky = new Dog (|)
```

```
Dog(string name, int age, int legsCount, string breed)
```

Ссылка: 5

```
abstract class Anima DirectoryNotFoundException
```

//Создание собаки

```
Dog husky = new Dog("Хаски", 4, 4, "Сибирский Хаски");  
husky.move();//вызов метода из родительского класса  
husky.barking();//вызов метода из самого класса
```

Ссылка: 5

Собака создана

Хаски передвигается на своих 4-х ногах

гав гав..

Абстрактные классы

Животное представляет некоторую абстракцию, которая не имеет конкретного воплощения. То есть есть коты, жирафы, обезьяны, рыбы, но как такового животного нет. Тем не менее все животные имеют нечто общее.

Но главное отличие абстрактных классов от обычных состоит в том, что мы **НЕ можем** создавать экземпляры абстрактного класса.